2ºProjeto AED

Daniel Oliveira, Rúben Franco
(115359) doliveira24@ua.pt, (114827) rrfranco@ua.pt

January 2024

1 Apresentação

Neste relatório, exploramos o desenvolvimento e a análise de algoritmos no contexto do Tipo Abstrato de Dados GRAPH, com ênfase em grafos orientados. Nosso objetivo principal é concluir o desenvolvimento do TAD GRAPH, implementar algoritmos eficientes para a determinação da ordenação topológica em grafos orientados e caracterizar a complexidade algorítmica dessas soluções.

Leitura de Grafos a partir de Arquivo:

Desenvolvemos uma função para ler grafos a partir de um arquivo, incluindo informações sobre direcionamento e ponderação.

Ordenação Topológica:

Na seção de Ordenação Topológica, implementamos três algoritmos distintos:
Algoritmo 1: Cria uma cópia do grafo e remove vértices sem arestas de entrada até a conclusão da ordenação.
Algoritmo 2: Utiliza um array para armazenar o in-degree de cada vértice, decrementando-o conforme os vértices são processados.
Algoritmo 3: Utiliza uma fila para armazenar vértices com in-degree zero, processando-os e atualizando os in-degrees dos vértices adjacentes.

Implementamos também funções para verificar invariantes do grafo e exibir grafos e ordenações topológicas de forma clara no console.

2 Análise da função 'GraphTopoSortComputeV1'

A função GraphTopoSortComputeV1 implementa um algoritmo para a ordenação topológica de um grafo dirigido. A abordagem adotada cria uma cópia do grafo original e, em seguida, itera sobre os vértices, removendo aqueles sem arestas de entrada até que todos os vértices tenham sido adicionados à sequência topológica.

A inicialização da estrutura GraphTopoSort é seguida por um loop que busca vértices sem arestas incidentes, removendo-os e atualizando a cópia do grafo. O algoritmo lida eficientemente com ciclos, encerrando a execução e indicando um erro se um ciclo é encontrado.

2.1 Complexidade

O algoritmo requer uma cópia do grafo, para esse fim é usado a função GraphCopy, que tem uma complexidade diretamente proporcional à soma do número de vértices com o número de arestas. Assumimos que o grafo é acíclico, então cada vértice do grafo vai ser encontrado numa pesquisa linear. A pesquisa linear têm a importante propriedade de parar uma, e uma só vez, a cada vértice do grafo. Logo a complexidadde efetiva da pesquisa linear quando executada no algoritmo é igual à sua complexidade média, que pode ser considerada constante. Depois de uma procura bem sucedida é necessário obter os vértices adjacentes aos vértices selecionados e removê-los. Os vértices adjacentes são obtidos pela função GraphGetAdjacentsTo. A complexidade desta função é diretamente proporcional ao outDegree do vértice que esta a ser processado. Depois é usada a função GraphRemoveEdge por cada vértice adjacente. A específica implementação de grafos neste projeto dá-nos a garantia que a ordem dos vértices adjacentes dada pela função GraphGetAdjacentsTo é a mesma ordem da lista de adjacências do vértice. A função GraphRemoveEdge é implementada de forma a procurar o vértice a remover com a mesma ordem da lista de adjacências. Com a garantia desta implementação, é possível determinar que a função GraphRemoveEdge tem uma complexidade efetiva constante, já que vai sempre remover o primeiro vértice adjacente da lista de adjacências, e só vai fazer isso com vértices que estão de certeza na lista de adjacências.

Sabendo isto, a complexidade da GraphTopoSortComputeV1 pode ser dada pela seguinte expressão:

$$(V + E) + \sum_{v=1}^{V} (1 + outDegree_v * 2)$$

Simplificando a fórmula, fica:

$$(V + E) + \sum_{v=1}^{V} outDegree_v$$

Onde V é o número de vértices, E é o número de arestas, $V + E$ é a complexidade da GraphCopy, $outDegree_v$ é o grau de saída do vértice V.

3 Análise da função 'GraphTopoSortComputeV2'

Esta implementação utiliza uma estratégia eficiente para encontrar a ordenação topológica, evitando a necessidade de criar uma cópia do grafo. Em vez disso, ela mantém um array auxiliar para armazenar os in-degrees de cada vértice.

O loop principal percorre todos os vértices até que todos tenham sido adicionados à ordenação topológica. Para cada iteração, um vértice com in-degree 0 e não marcado é identificado, e sua remoção é simulada, ajustando os in-degrees dos vértices adjacentes.

3.1 Complexidade

O 2ºalgoritmo é bastante similar ao primeiro, em vez de usar uma cópia do grafo é usado um array auxiliar de complexidade de inicialização diretamente proporcional ao número de vértices. Em vez das arestas serem removidas por GraphRemoveEdge, apenas é decrementado ao grau de entrada dos vértices adjacentes do array auxiliar.

A expressão de complexidade será:

$$V + \sum_{v=1}^{V} (1 + outDegree_v * 2)$$

Simplificando a fórmula, fica:

$$V + \sum_{v=1}^{V} outDegree_v$$

4 Análise da função 'GraphTopoSortComputeV3'

Esta função inicia garantindo que o grafo passado como argumento não seja nulo e é um digrafo, assegurando a validade das operações subsequentes. Em seguida, ela cria e inicializa uma estrutura GraphTopoSort que será usada para armazenar a sequência topológica e outras informações relevantes.

A função utiliza uma fila (candidates) para gerenciar os vértices candidatos para a ordenação topológica, inicializando a fila com os vértices de grau de entrada zero. Em seguida, ela entra em um loop principal que processa os vértices, enfileirando aqueles cujo grau de entrada se torna zero durante o processo.

O grau de entrada é eficientemente atualizado à medida que os vértices são processados, e a função verifica se todos os vértices foram processados com sucesso, indicando assim uma ordenação topológica válida.

4.1 Complexidade

O algoritmo utiliza um array auxiliar com o grau de entrada dos vértices, então tem uma complexidade de inicialização diretamente proporcional ao número de vértices do grafo. O algoritmo começa por examinar todos os vértices e os que tem grau de entrada zero são enfileirados. É iniciado um loop que não termina até a fila auxiliar esteja vazia, o que num grafo acíclico só acontece quando todos os vértices são processados. Dentro do loop o vértice é adicionado à sequência

topológica, e o grau de entrada no array auxiliar dos vértices adjacentes é decrementado. Todos os vértices adjacentes que depois desta operação tenham grau de entrada zero são enfileirados para depois serem processados.

A expressão de complexidade será:

$$V + \sum_{v=0}^{V} outDegree_v * 2$$

Simplificando a fórmula fica:

$$V + \sum_{v=0}^{V} outDegree_v$$